

Machine Learning and Truck Platooning

S M Mahmud Hasan,¹ Talha Khan

Contents

1	Motivation	2
2	Requirements and SysML Models	2
2.1	Scenario A - Membership Change	2
2.2	Scenario B - Communication Loss	3
3	SVM Approach	5
4	Dataset and Evaluation	5
5	Combined UPPAAL Model	6
6	Results and Discussion	8
7	Limitations	8
8	Conclusion	9
9	Declaration of Originality	9

Abstract: This project contains two parts: an SVM-based line-following task and UPPAAL models for truck platooning. In the first part, a camera image is processed with OpenCV to detect a green guide line, and a Support Vector Machine (SVM) classifies the steering direction. With an RBF-kernel SVM and an 80/20 stratified split of 2,241 samples (450 binary mask features each), the model achieved 91.76% accuracy on 449 test samples. In the second part, we modeled two platooning scenarios: controlled joining and leaving of the platoon (Scenario A) and communication loss between a following truck and the lead truck (Scenario B). SysML diagrams describe the requirements and behavior of both scenarios, and a single combined UPPAAL model integrates membership changes, gap closing, and heartbeat-based communication-loss handling. The UPPAAL verifier confirms deadlock freedom, requirement-derived timing invariants, and reachability of all main scenario states.

¹ Department of Electronic Engineering (ELE), Hamm-Lippstadt University of Applied Sciences (HSHL), Lippstadt, Germany. s-m-mahmud.hasan@stud.hshl.de

1 Motivation

Autonomous driving combines real perception problems with safety-related behavior. This project addresses two such problems: Steering a line-following vehicle from camera images, and describing how a truck platoon manages trucks joining and leaving, and how the system reacts when communication between the lead truck and a following truck becomes delayed or unavailable. We used machine learning for the camera-based steering task and SysML modeling for the platooning scenarios.

For the line-following task, our model recognizes a green line in camera frames and produces a steering value in the interval $[-1.0, 1.0]$. The platooning tasks cover a truck that joins or leaves an active platoon under operator supervision (Scenario A), and a following truck that loses communication with the lead truck and must increase its safety distance, notify other actors, reconnect if possible, or leave the platoon safely (Scenario B). The timing aspects of these tasks make them suitable for timed automata and UPPAAL verification [AD94; BDL04].

The goal was not to build a production-ready vehicle, but to document the implemented methods and the test results, plus the limitations. The repository used for this report includes a ROS 2 framework, the implemented code with OpenCV image processing, data-collection and SVM-training scripts, saved feature and label arrays with a trained model, SysML diagrams, and the combined UPPAAL model.

2 Requirements and SysML Models

2.1 Scenario A - Membership Change

Scenario A covers controlled joining and leaving of trucks under operator supervision. Its main requirement is that the system shall safely manage membership changes in an active platoon without disrupting the remaining members (REQ-1.0). The requirements diagram in Fig. 1 derives six requirement groups from this goal: safe distance, communication, speed synchronization, safety validation, member notification, and gap management. Each of these is refined with sub-requirements such as a speed tolerance of ± 0.5 m/s, V2X data exchange at a minimum of 50 Hz, membership notifications within 100 ms, and gap closure within 5 s.



Fig. 1: Requirements diagram for Scenario A.

The state machine in Fig. 2 gives the local behavior of a joining or leaving truck. From Idle, the composite Join Procedure validates conditions, establishes the V2V link, and

synchronizes speed before the truck enters Active Platooning, where the platoon acts as one coordinated system and closes spacing gaps when a predecessor leaves. The Leave Procedure validates and executes a controlled exit, and a Link Reconnection Protocol serves as fallback on link failure: safe deceleration, operator notification, and either reconnection or a safe decouple.

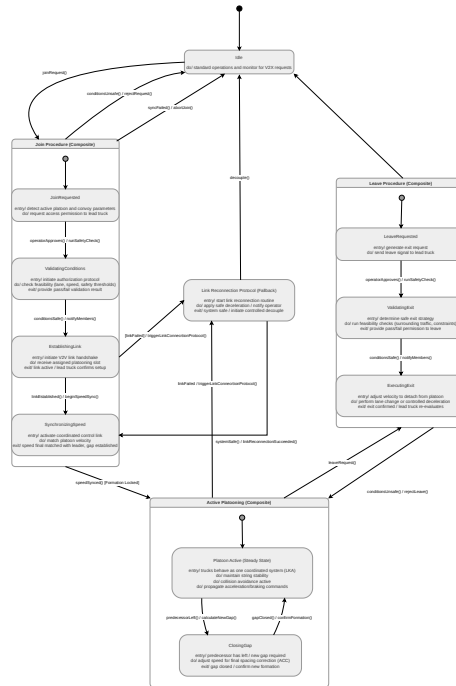


Fig. 2: State-machine model of the joining/leaving truck in Scenario A.

2.2 Scenario B - Communication Loss

Scenario B looks at what happens when connection between a follower truck and the lead truck drops and the system stops getting reliable messages. Fig. 3 shows the requirements it has to meet: detect the missing messages, increase its distance, and then either reconnect or leave the platoon safely.

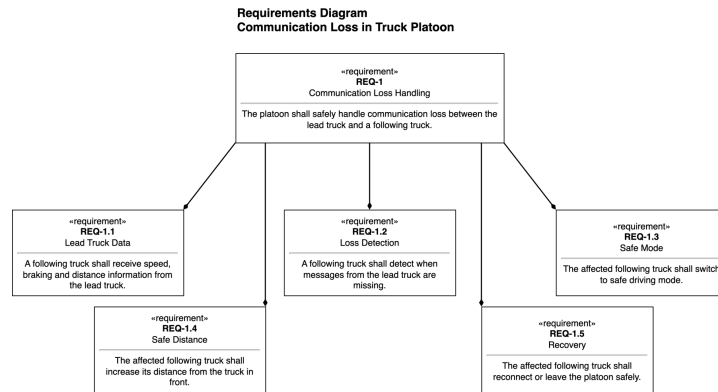


Fig. 3: Requirements diagram for safe communication-loss handling.

The sequence model (Fig. 4) shows the order of actions. Normal data transfer, timeout detection, safe-mode activation, warning, reconnection, and either rejoining or safe leaving, while the state machine (Fig. 4) gives the local behavior of the affected truck from normal platoon driving to standalone driving.

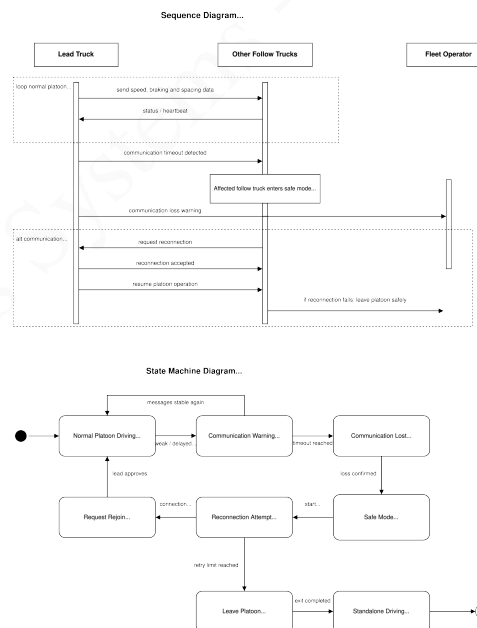


Fig. 4: Communication-loss handling model consisting of a sequence diagram and state machine.

3 SVM Approach

The camera pipeline crops the lower 60% of the 720×1280 BGR frame, thresholds green pixels in HSV space, and cleans the binary mask with 7×7 elliptical closing and opening; the largest contour is treated as the line. The mask is then resized to 30×15 pixels and flattened into a 450-dimensional feature vector. The target steering value is generated from the contour centroid using

$$\text{offset} = \frac{x_{\text{line}} - x_{\text{center}}}{x_{\text{center}}}, \quad \text{steer} = \text{clip}(1.5 \cdot \text{offset}, -1, 1), \quad (1)$$

and discretized into three classes: left below -0.10 , straight between -0.10 and 0.10 , and right above 0.10 .

The classifier is `sklearn.svm.SVC` with an RBF kernel ($C = 5.0$, $\gamma = 0.05$), balanced class weights, probability output enabled, and random state 42; the RBF kernel was chosen because the steering classes are not separated by a simple linear boundary in the mask feature space [CV95; Pe11]. At runtime, class probabilities are converted into a smoother steering command,

$$\text{steering} = P(\text{right}) - P(\text{left}), \quad (2)$$

which avoids the abrupt transitions of a direct mapping to -1 , 0 , and $+1$. If the model cannot be loaded or prediction fails, the implementation falls back to a contour-centroid heuristic.

4 Dataset and Evaluation

The dataset contains 2,241 samples with 450 features per sample. The training script uses an 80/20 stratified split with random state 42, giving 1,792 training and 449 test samples while keeping the class proportions stable (Table 1).

Tab. 1: Dataset and split used for SVM evaluation.

Class	Label	Total	Test set
Left	-1	574	115
Straight	0	801	160
Right	+1	866	174
Total		2,241	449

Tab. 2: SVM test-set classification report.

Class	Precision	Recall	F1	Support
Left	0.90	0.92	0.91	115
Straight	0.95	0.86	0.90	160
Right	0.91	0.97	0.94	174
Accuracy	91.76% over 449 test samples			

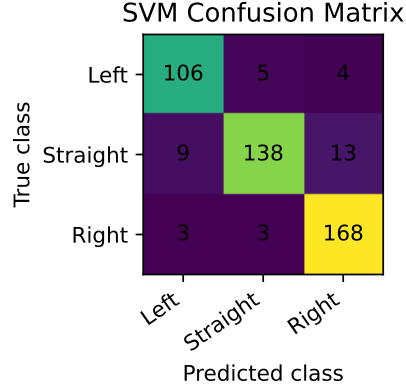


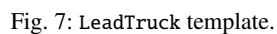
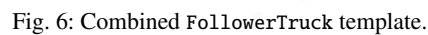
Fig. 5: Confusion matrix for the stratified test split.

Most errors in the confusion matrix (Fig. 5) come from the straight class, which lies close to the boundary between small left and right deviations, so some samples are predicted as slight corrections.

5 Combined UPPAAL Model

Both scenarios are verified in a single combined UPPAAL model with three templates: `FollowerTruck` (instantiated as `truck0` and `truck1`), `LeadTruck`, and `SafetyOperator`. The follower template (Fig. 6) implements the Scenario A state machine directly by utilizing the three main processes: `JOINING`, `ACTIVE_PLATOONING`, and `LEAVING`. Each process is bounded by its own clock. Safety validation must finish within `NOTIFICATION_LATENCY` (100 ms, REQ-1.5.1), and speed synchronization within `SYNC_TIMEOUT` (3 s, REQ-1.3.2), where the deviation stays at ± 0.5 m/s (REQ-1.3.1). During steady platooning a V2X exchange happens every `V2X_CYCLE` (20 ms, or 50 Hz, REQ-1.2.3). When a predecessor leaves, the remaining truck closes the gap within `GAP_CLOSE_TIMEOUT` (5 s, REQ-1.6.2). The lead truck (Fig. 7) tracks the member count, routes join and leave requests through the safety operator (Fig. 8), coordinates the gap-closing handshake, and broadcasts a periodic heartbeat to all members.

Scenario B is merged into the same follower template as a watchdog on the steady platooning state. If a member misses three consecutive heartbeats (`COMM_TIMEOUT = 3 \cdot V2X_CYCLE`), it raises `commWarning` and enters `COMM_LinkLost` then falls back to independent safe driving within `SAFE_MODE_DELAY`. Afterwards, it either rejoins on a received heartbeat inside the reconnection window (`RECONNECT_LIMIT`) or gives up and exits the platoon entirely via `commGiveUp`. Two shared flags, `commActive` and `leaveInProgress`, make sure that communication-loss handling and a coordinated voluntary leave are mutually exclusive.



of the frame-level probabilities, so the steering output is only as smooth as the per-frame predictions.

The UPPAAL model has its own simplifications. Vehicle dynamics are not modeled at all. The automata only capture coordination and timing. The lead coordinates one maneuver at a time, and the reconnection window is an abstract bound rather than a value derived from distance or speed.

8 Conclusion

This project combines an OpenCV-based, 450-feature RBF-SVM line follower with SysML models and a combined, fully verified UPPAAL model for truck-platoon membership change and communication loss. It demonstrates practical ML evaluation and formal timing analysis within one autonomous-systems workflow.

9 Declaration of Originality

We, S M Mahmud Hasan and Talha Khan, declare that this paper is our own work and that only the cited sources and aids were used. Direct quotations are marked, and all borrowed statements and ideas are fully referenced. This work has not been submitted to any examination body in the same or similar form and has not been published. We agree that our work may be checked by a plagiarism checker.

Lippstadt, July 10, 2026 - S M Mahmud Hasan, Talha Khan

References

- [AD94] Alur, Rajeev; Dill, David L.: A theory of timed automata. Theoretical Computer Science 126(2), pp. 183–235, 1994.
- [BDL04] Behrmann, Gerd; David, Alexandre; Larsen, Kim G.: A tutorial on UPPAAL. In: Formal Methods for the Design of Real-Time Systems. Springer, pp. 200–236, 2004.
- [CV95] Cortes, Corinna; Vapnik, Vladimir: Support-vector networks. Machine Learning 20, pp. 273–297, 1995.
- [Pe11] Pedregosa, Fabian; Varoquaux, Gael; Gramfort, Alexandre; Michel, Vincent; Thirion, Bertrand; Grisel, Olivier; Blondel, Mathieu; Prettenhofer, Peter; Weiss, Ron; Dubourg, Vincent; Vanderplas, Jake; Passos, Alexandre; Cournapeau, David; Brucher, Matthieu; Perrot, Matthieu; Duchesnay, Edouard: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, pp. 2825–2830, 2011.